

Product Requirements Document

ShopEasy — E-Commerce Web Application

Frontend Assignment

Document Version	1.0
Status	Draft
Owner	Samidha Powar
Last Updated	05 July 2026
Target Release	Assignment Submission
Document Type	Product Requirements Document (PRD)

1. Executive Summary

ShopEasy is a modern, responsive e-commerce web application that enables users to browse a product catalog, search and filter products, manage a shopping cart and wishlist, and complete a mock checkout flow. Built as a frontend-only application against the public DummyJSON API, the product demonstrates production-grade architecture patterns including feature-sliced organization, centralized state management with Redux Toolkit, and environment-based configuration.

The application is designed to run standalone (no backend service required) while adhering to conventions used in production e-commerce frontends such as Amazon, Flipkart, and Myntra. It uses mock authentication, mock payment simulation, and persistent client-side state to deliver a realistic end-to-end shopping experience.

1.1 Purpose

This document defines the scope, functional requirements, non-functional requirements, technical architecture, and acceptance criteria for the ShopKart application. It serves as the single source of truth for what will be built, why it will be built, and how success will be measured.

1.2 Scope

In scope:

- Product browsing, search, filtering, sorting, and pagination
- Product detail views with image gallery and product metadata
- Cart and wishlist management with client-side persistence
- Mock authentication using DummyJSON credentials
- Multi-step checkout flow with form validation
- Simulated payment processing and order confirmation
- Responsive design supporting mobile, tablet, and desktop viewports

Out of scope:

- Real payment gateway integration (Razorpay/Stripe production keys)
- Backend services, databases, or server-side logic
- Order fulfillment, inventory management, or shipping integrations
- Admin panel or seller dashboards
- Real user account persistence across devices
- Email/SMS notifications

2. Problem Statement and Goals

2.1 Problem Statement

Modern users expect e-commerce interfaces to be fast, intuitive, and consistent across devices. Frontend developers building such interfaces need a reference implementation that demonstrates industry-standard patterns for state management, API integration, form handling, and modular architecture. Existing tutorials

and starter kits often lack production considerations such as environment separation, error boundaries, persistent state, and feature-sliced organization.

2.2 Goals

- Deliver a functional, end-to-end shopping experience powered entirely by the frontend and a public mock API
- Demonstrate production-ready patterns: feature-sliced architecture, centralized state, environment-based configuration, and error handling
- Achieve a smooth user journey from product discovery to order confirmation in under 60 seconds
- Ensure full responsiveness across viewports from 320px (mobile) to 1440px+ (desktop)
- Maintain a clean, maintainable codebase suitable for extension by additional engineers

2.3 Non-Goals

- This is not a production e-commerce system; it will not handle real transactions or PII
- This is not a design system project; visual design is functional, not exhaustive
- This is not a performance benchmark; targets are pragmatic (see Section 7)

3. Target Users and Personas

3.1 Primary Persona: The Casual Shopper

Profile: Age 22–45, uses mobile primarily, occasionally shops on desktop. Comfortable with e-commerce platforms like Amazon, Flipkart, and Myntra. Expects fast load times, clear product images, transparent pricing, and a frictionless checkout.

Key needs: Browse products by category, filter by price and rating, add items to cart quickly, save items for later, complete checkout with minimal friction.

3.2 Secondary Persona: The Deliberate Buyer

Profile: Comparison shopper who researches products before purchasing. Uses wishlist heavily. Cares about product details, ratings, and descriptions.

Key needs: Detailed product pages, ability to save items across sessions, easy comparison of similar products, clear pricing including discounts.

4. Functional Requirements

4.1 Product Discovery

FR-1: Product Listing

- System shall display a paginated grid of products on the home and products routes
- Each product card shall display: thumbnail, title, price, discount percentage, rating, and stock indicator
- System shall show loading skeletons while product data is being fetched

- System shall show a meaningful empty state when no products match the current filters
- System shall show an error state with retry option on API failure

FR-2: Search

- System shall provide a search input in the primary navigation
- Search input shall debounce user input by 300ms before triggering a query
- Search results shall replace the current product listing
- Empty search results shall display an empty state with suggestions to clear filters

FR-3: Filtering and Sorting

- Users shall be able to filter products by category
- Users shall be able to sort products by price (ascending/descending) and rating
- Filter and sort state shall reflect in the URL as query parameters for shareability
- Users shall be able to clear all filters with a single action

FR-4: Product Detail

- Each product shall have a dedicated detail page accessible at /products/:id
- Detail page shall display: image gallery with thumbnails, title, brand, price, discount, rating, description, stock status, category, and quantity selector
- Detail page shall provide primary actions: Add to Cart and Add to Wishlist
- Detail page shall handle invalid product IDs with a 404 view

4.2 Cart Management

FR-5: Cart Operations

- Users shall be able to add products to the cart from the product listing and detail pages
- Users shall be able to view the cart via a slide-in drawer or dedicated cart page
- Users shall be able to increment, decrement, or directly edit the quantity of any cart item
- Users shall be able to remove individual items or clear the entire cart
- The cart shall display: item subtotals, cart subtotal, applicable discounts, and final total
- The cart badge in the navigation shall reflect the total item count in real-time
- The cart state shall persist across browser sessions using client-side storage

4.3 Wishlist Management

FR-6: Wishlist Operations

- Users shall be able to add or remove products from a wishlist
- Users shall be able to view the wishlist on a dedicated page
- Users shall be able to move items from the wishlist to the cart
- The wishlist state shall persist across browser sessions

4.4 Authentication

FR-7: Login

- System shall provide a login page at /login
- System shall authenticate users against the DummyJSON /auth/login endpoint
- On successful authentication, the returned token shall be persisted for subsequent sessions
- Login failure shall display a clear inline error message
- Authenticated users shall be redirected to their originally intended destination after login

FR-8: Session Management

- Auth state shall persist across page reloads and browser restarts until explicit logout
- Logout shall clear the auth token, cart, wishlist, and any order history from client storage
- Protected routes shall redirect unauthenticated users to the login page

4.5 Checkout**FR-9: Checkout Flow**

- Access to /checkout shall require an authenticated session
- Checkout shall be a multi-step flow: (1) shipping address, (2) payment details, (3) review and place order
- All form inputs shall be validated using react-hook-form with zod schemas
- Validation errors shall be displayed inline immediately on field blur
- Users shall be able to navigate backward through steps without losing entered data

FR-10: Payment Simulation

- Payment step shall accept card details: number, expiry, CVV, cardholder name
- Card number shall be validated using the Luhn algorithm
- On submit, system shall display a loading state for 2 seconds simulating payment processing
- Payment shall always succeed for the purposes of this application (no failure simulation required)
- On success, the cart shall be cleared and the user redirected to the order confirmation page

FR-11: Order Confirmation

- Order confirmation page shall display: order ID, items purchased, quantities, prices, total paid, shipping address, and estimated delivery date
- Order shall be added to the user's order history
- Order confirmation shall be shareable via URL (order ID in path)

5. Technical Architecture**5.1 Technology Stack**

Layer	Technology
Framework	React 18 (JavaScript, no TypeScript)
Build Tool	Vite
Routing	React Router v6

State Management	Redux Toolkit with createSlice pattern
Data Fetching	RTK Query
Persistence	redux-persist (localStorage backend)
Forms	react-hook-form + zod resolver
Notifications	react-toastify
Styling	Plain CSS with CSS custom properties (design tokens)
API Source	DummyJSON (https://dummyjson.com)

5.2 Architectural Pattern

The application follows a feature-sliced architecture. Each business domain (products, cart, wishlist, auth, checkout, orders) lives in its own directory under `src/features/` and contains all files related to that domain: components, Redux slice, API endpoints, and helpers. Each feature exposes a public API via an `index.js` barrel file, controlling what other features may import.

This pattern was chosen over the traditional type-sliced organization (`components/`, `reducers/`, `actions/`) because it scales better as the application grows, keeps related code colocated, and reduces cross-feature coupling.

5.3 Folder Structure

The application uses a flat, feature-sliced structure:

- `src/assets/` — static assets (images, icons, fonts)
- `src/components/` — generic reusable UI primitives (Button, Input, Modal, Skeleton, EmptyState, ErrorBoundary, Navbar, Footer)
- `src/config/` — environment configuration and constants
- `src/features/` — feature slices: auth, products, cart, wishlist, checkout, orders
- `src/hooks/` — global reusable hooks (useDebounce, useMediaQuery)
- `src/layouts/` — page frame components (MainLayout)
- `src/routes/` — routing configuration, protected route wrapper, and route path constants
- `src/stores/` — Redux store configuration, root reducer, persist configuration, RTK Query base API, and cross-cutting common slice/actions
- `src/styles/` — plain CSS files organized by feature, imported via a single `main.css` entry point
- `src/utils/` — formatters and helper utilities

5.4 State Management Strategy

State is partitioned into three categories, each managed with the appropriate tool:

- **Server state** (products, categories, auth token issuance) is managed by RTK Query with automatic caching, invalidation, and re-fetching.
- **Client state** (cart items, wishlist items, authenticated user, orders) is managed by Redux Toolkit slices and persisted to `localStorage` via `redux-persist`.
- **Ephemeral UI state** (drawer open/closed, theme, mobile menu state) is managed by a shared `commonSlice` in the `stores/common/` directory.

5.5 Environment Configuration

The application supports three environments: development, nonprod, and production. Each environment has its own .env file at the project root:

- .env.development — used by npm run dev
- .env.nonprod — used for staging-like builds and previews
- .env.production — used for production builds

Environment variables are prefixed with VITE_ to be exposed to the client bundle. All environment access flows through src/config/env.js, which validates required variables at startup and exposes helpers such as isDev, isNonProd, and isProd.

6. User Experience Requirements

6.1 Responsive Design

- The application shall be fully responsive from 320px (mobile) to 1440px+ (desktop)
- Mobile-first CSS approach: base styles target mobile, media queries progressively enhance for larger screens
- Standard breakpoints: 640px (sm), 768px (md), 1024px (lg), 1280px (xl)
- Touch targets on mobile shall be at least 44x44 pixels

6.2 Loading and Empty States

- All data-fetching views shall display skeleton loaders during load, not raw spinners
- Empty states shall include an illustration or icon, a headline, and a clear call to action
- Error states shall include a headline, a friendly message, and a retry button where applicable

6.3 Feedback and Notifications

- Cart additions, wishlist changes, login success, logout, and checkout completion shall trigger toast notifications
- Destructive actions (clear cart, remove all wishlist items) shall show a confirmation modal
- Form validation errors shall appear inline below the affected field

6.4 Accessibility

- All interactive elements shall be keyboard-navigable
- All images shall have descriptive alt text
- Color contrast shall meet WCAG AA standards for body text
- Form fields shall have associated labels
- ARIA attributes shall be used where semantic HTML is insufficient

7. Non-Functional Requirements

7.1 Performance

- Initial page load (First Contentful Paint) shall be under 2.5 seconds on a 4G connection
- Route transitions shall feel instantaneous (under 100ms) for cached routes
- Search input shall debounce to avoid excessive API calls
- Images shall be lazy-loaded below the fold

7.2 Reliability

- The application shall handle DummyJSON API failures gracefully with user-facing error states
- An `ErrorBoundary` shall wrap the application root to catch unexpected render errors
- Client-side persistence failures shall not crash the application

7.3 Maintainability

- Code shall follow feature-sliced organization
- Each feature shall expose a single public API via its `index.js`
- Redux slices shall use `createSlice`; no legacy switch-statement reducers
- Environment access shall be centralized in `src/config/env.js`
- Route paths shall be centralized in `src/routes/routePaths.js`

7.4 Browser Support

- Latest two versions of Chrome, Firefox, Safari, and Edge
- No IE11 support required

8. Success Metrics

The application will be considered successful if it meets the following criteria:

- A first-time user can complete the full flow (browse → detail → add to cart → login → checkout → confirmation) in under 60 seconds
- Cart and wishlist state survives a full browser refresh without data loss
- The application renders correctly at 375px, 768px, and 1440px viewport widths
- All primary flows work without console errors
- Lighthouse Performance score is 80 or higher on desktop

9. Assumptions, Dependencies, and Risks

9.1 Assumptions

- DummyJSON API remains available and stable for the duration of the assignment
- Users have modern browsers with JavaScript enabled
- Users have `localStorage` available and enabled

9.2 Dependencies

- External: DummyJSON public API
- Internal: no backend services required

9.3 Risks and Mitigations

Risk	Impact	Mitigation
DummyJSON API downtime	High — app becomes unusable	Display friendly error state; document known limitation in README
DummyJSON write endpoints do not persist	Medium — cart sync would fail	Handle cart/wishlist client-side only via redux-persist
Timeline pressure (2-day build window)	Medium — scope creep	Ship MVP first: browse, detail, cart. Add wishlist, auth, checkout iteratively
localStorage disabled by user	Low — persistence would fail silently	Gracefully fall back to in-memory state; wrap persist logic in try/catch

10. Acceptance Criteria

The application is complete and ready for submission when all of the following are true:

- All functional requirements in Section 4 are implemented
- All non-functional requirements in Section 7 are met
- The application runs successfully via `npm run dev` on a clean install
- README.md documents the project overview, setup, environment variables, scripts, folder structure, and design decisions
- No console errors during the primary user flow
- Cart, wishlist, and auth state persist across browser refreshes
- Application is responsive across the specified viewport range

11. Open Questions

- Should the checkout flow include an order summary sidebar visible across all steps, or only in the review step?
- Should the wishlist and cart be merged into a single "Saved Items" view, or remain distinct?
- Should dark mode be included in the initial release or deferred?
- Should the application include a compare-products feature in a future iteration?

12. Appendix

12.1 Glossary

- **RTK Query:** Redux Toolkit's data-fetching and caching solution, built on top of Redux.
- **Feature-sliced architecture:** A code organization pattern where files are grouped by business domain rather than by file type.
- **Design tokens:** Named CSS variables representing design decisions such as colors, spacing, and typography scale.

- **Debounce:** A technique to delay function execution until a specified time has passed since the last invocation, used to reduce API calls during rapid user input.

12.2 References

- DummyJSON API documentation: <https://dummyjson.com/docs>
- Redux Toolkit documentation: <https://redux-toolkit.js.org>
- React Router documentation: <https://reactrouter.com>
- Vite documentation: <https://vitejs.dev>